

Table of Contents

Week 5 — Constructors & Object Initialization

Ensuring Every Object Is Complete and Valid From the Moment It Exists

Lesson Overview

Hour Plan

Recap of Week 4 (5 min)

Part 1 — The Problem: Incomplete Objects (13 min)

Open with a story (3 min)

The Three Problems With Incomplete Setup

The Goal

Part 2 — What Is a Constructor? (12 min)

Open with a question (2 min)

Definition

The Hotel Check-In Analogy in Detail

Three Key Properties of a Constructor

Connecting to Encapsulation (Week 4)

Part 3 — Flexible Constructors: Defaults and Overloading (10 min)

Open with a question (2 min)

Defaults — When Some Information Has a Sensible Fallback

Overloading — Multiple Setup Options for the Same Object

The Key Rule for Defaults

Part 4 — The Master Constructor Principle (10 min)

The Problem With Copying Defaults

The Master Constructor — One Source of Truth

Why the Master Constructor Matters

Chaining — How One Constructor Calls Another

The Critical Rule for Chaining

Part 5 — Java as Illustration (8 min)

Reading a Constructor as a Concept

Reading as Plain English

The Before and After

Test Checkpoint

Common Misconceptions

- Basic — Spot the Problem
- Intermediate — Design the Setup Paths
- Advanced — Constructor Design Decisions

Lesson Summary

Homework

Week 5 — Constructors & Object Initialization

Ensuring Every Object Is Complete and Valid From the Moment It Exists

Date: Wednesday, 1 July 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Constructors — building complete, valid objects in a single step
Date & Time	Wednesday, 1 July 2026 · 6:30 PM – 8:30 PM
Duration	2 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 4 — Encapsulation, private attributes, getters, setters, validation
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain what a constructor is and why it exists, describe the problem it solves using real-world analogies, understand how defaults and overloading provide flexibility, understand the master constructor principle, and trace how validation still applies during object creation

Hour Plan

00:00 – 00:05	Recap of Week 4 + Homework check
00:05 – 00:18	Part 1 – The Problem: Incomplete Objects (13 min)
00:18 – 00:30	Part 2 – What Is a Constructor? (12 min)
00:30 – 00:40	Part 3 – Flexible Constructors: Defaults and Overloading (10 min)
00:40 – 00:50	Part 4 – The Master Constructor Principle (10 min)
00:50 – 00:58	Part 5 – Java as Illustration (8 min)
00:58 – 01:00	Checkpoint + Homework

Recap of Week 4 (5 min)

Ask students verbally:

1. "What is encapsulation in one sentence? What problem does it solve?"
→ *Hiding an object's data and controlling access through approved actions.*
2. "What is the difference between a getter and a setter? Why does validation belong in the setter?"
→ *Getter reads; setter writes with a check. Validation in the setter means no bad value can sneak in.*
3. "From homework — you made all Smartphone attributes private and added a charging action with a battery limit. What happens if someone tries to set the battery to 500%?"
→ *The setter's rule rejects it.*
4. Transition question: "After making everything private, how many separate steps did it take to create one complete Smartphone object?"
→ *At least 3-4 setter calls after creating it. Was this convenient? What if you forgot one?*

That inconvenience is exactly what today's lesson solves.

Part 1 — The Problem: Incomplete Objects (13 min)

Open with a story (3 min)

"Imagine you arrive at a hotel. The receptionist hands you a room key immediately — before you've given your name, shown your ID, or paid anything.

You go to your room, open the door, and find the bed isn't made, there are no towels, and the previous guest's belongings are still there.

Would you call this a 'valid' hotel stay? Could the hotel operate like this reliably?"

Let students react. They will immediately recognize the absurdity — there's no accountability, no record, and the room isn't ready.

"This is exactly what happens when we create an object and leave it in an incomplete state. We handed over the key before the room was ready."

The Three Problems With Incomplete Setup

After learning encapsulation in Week 4, the sequence to create one valid Student object looks like this:

THE CURRENT, AWKWARD SEQUENCE:

Step 1: Create the object → `Student sokha = new Student()`
 Object exists but is EMPTY
 name = ???, age = ???, major = ???

Step 2: Set the name → `sokha.setName("Sokha")`

Step 3: Set the age → `sokha.setAge(20)`

Step 4: Set the major → `sokha.setMajor("Computer Science")`

Step 5: Now it is finally valid and usable

What if someone stops at Step 2?

→ Sokha's age = 0, major = nothing

→ An incomplete, invalid student record exists in the system

Three specific things go wrong:

PROBLEM 1 – INCOMPLETE OBJECTS CAN EXIST

Nothing forces the creator to finish the setup.

A student with no age and no major is meaningless –
but the system holds it anyway, without complaint.

PROBLEM 2 – NOTHING ENSURES THE RIGHT ORDER

What if major should always be set before GPA?

The creator has no way of knowing – the class doesn't enforce it.
One wrong order and the object starts life corrupted.

PROBLEM 3 – THE SAME SETUP IS REPEATED EVERYWHERE

Every time a new Student is created anywhere in the program,
the same 3-4 setup calls must be repeated.

A new required field added later breaks every creation point.

The Goal

THE GOAL:

Create an object that is COMPLETE and VALID in a single step.
The class itself – not the caller – controls how it is built.

Before: `Student sokha = new Student()` ← empty, invalid

```
sokha.setAge(20)
    sokha.setMajor("CS")
```

After: `Student sokha = new Student("Sokha", 20, "CS")`
← complete, valid, ready in one step

Part 2 — What Is a Constructor? (12 min)

Open with a question (2 min)

"Back to the hotel. How does a GOOD hotel check-in actually work? What does the receptionist collect before they give you a key?"

Students will say: name, ID, credit card, how many nights, sometimes a deposit. All upfront, all at once.

"The receptionist does not give you a key halfway through check-in. They collect everything needed to set up a complete, valid, ready room — AND THEN give you the key.

*This is what a **constructor** does for an object."*

Definition

A **constructor** is a special setup action that runs automatically the moment an object is created.

Its job is to collect all the information needed and build a complete, valid object — before anything else can happen.

The object is not available for use until the constructor finishes its work.

The Hotel Check-In Analogy in Detail

HOTEL RECEPTIONIST – A CONSTRUCTOR IN REAL LIFE

WHAT THE RECEPTIONIST (constructor) DOES:

- Collects all required information upfront (parameters)
- Checks each piece of information is valid (validation)
 - "Is this ID genuine?" "Is this card valid?"
- Sets up the room using that information

WHAT THE RECEPTIONIST DOES NOT DO:

- Hand over a key to an un-prepared room
- Accept check-in without a name or payment
- Let the guest set up their own room after arriving

THE RESULT:

- The moment you have the key (the object), the room is ready
 - No half-ready state exists – it is valid or you don't get in
-

Three Key Properties of a Constructor

PROPERTY 1 – IT RUNS AUTOMATICALLY

You do not call a constructor manually after creating an object. The moment you say "create a new Student", the constructor runs on its own and finishes BEFORE the object is handed to you.

PROPERTY 2 – IT RUNS EXACTLY ONCE PER OBJECT

It runs when the object is born. It never runs again. This is not a method you call repeatedly – it is a one-time setup event, like a hotel check-in.

PROPERTY 3 – THE OBJECT IS NOT READY UNTIL IT FINISHES

Like the hotel room not being "yours" until check-in is done, the object is not available until the constructor completes. This guarantees: if you have an object, it was set up correctly.

Connecting to Encapsulation (Week 4)

"Last week we learned that validation belongs in setters — so bad values cannot be stored. A constructor takes this one step further: it calls those same setters during setup, so the object starts life with already-validated information."

CONSTRUCTOR + ENCAPSULATION WORKING TOGETHER

Week 4 lesson:

"A setter checks a value before saving it.
age cannot be -5 because the setter rejects it."

Week 5 lesson (today):

"The constructor calls those same setters

So the very first value stored is already validated.
An invalid age of -5 gets corrected at birth –
the object never holds a bad value, not even for a second."

Part 3 — Flexible Constructors: Defaults and Overloading (10 min)

Open with a question (2 min)

"Same hotel. Two different types of guests arrive.

Guest A has a full reservation: name, ID, credit card, 3 nights, room preference — everything.

Guest B is a walk-in with no reservation: name and credit card only. The hotel handles the rest with sensible defaults: a standard room, 1 night, standard breakfast.

Both guests leave check-in with a valid, ready room. They just provided different amounts of information to get there."

Defaults — When Some Information Has a Sensible Fallback

Some attributes have a logical default value when not explicitly provided. A constructor can fill these in automatically.

STUDENT OBJECT – DEFAULTS FOR MISSING INFORMATION

If a student enrolls but hasn't declared a major yet:

→ major defaults to "Undeclared" (a meaningful, valid value)

If a student's age was not provided:

→ age defaults to 18 (a reasonable starting value)

If no GPA information exists yet:

→ gpa defaults to 0.0 (not yet assessed – valid)

These defaults ensure the object is ALWAYS in a valid state
even when the caller provides only partial information.

Overloading — Multiple Setup Options for the Same Object

*"The hotel offers three check-in paths — but they all produce the same result: a valid, ready room. This is **constructor overloading**: different ways to provide setup information, each producing the same type of valid object."*

STUDENT – THREE CHECK-IN PATHS (constructor overloading)

PATH A – Full details provided

"Sokha, age 20, Computer Science"

→ All three fields set from what was given

PATH B – Name and major only

"Dara, Engineering"

→ name = "Dara", major = "Engineering", age = 18 (default)

PATH C – Name only

"Chenda"

→ name = "Chenda", major = "Undeclared" (default), age = 18 (default)

ALL THREE PATHS produce a valid Student object.

The caller chooses the path that matches what they know.

The Key Rule for Defaults

A default must be a MEANINGFUL, VALID value – not just "nothing."

GOOD DEFAULT: age = 18 (makes sense as a starting age)

GOOD DEFAULT: major = "Undeclared" (truthful – not yet chosen)

BAD DEFAULT: age = 0 (a student aged 0 is meaningless)

BAD DEFAULT: name = "" (an empty name is invalid)

Part 4 — The Master Constructor Principle (10 min)

The Problem With Copying Defaults

"Imagine the hotel has three check-in paths. At the end of each, the receptionist fills in a master Guest Record form — the same form, every time.

If they write the same default 'standard room' on three separate forms by hand, and management later decides all defaults should be 'deluxe room' — the receptionist has to find and change all three forms separately. One might get missed.

The smarter approach: one receptionist writes the master form, and the other two paths just hand their information to HER."

The Master Constructor — One Source of Truth

All setup logic lives in ONE place — the master constructor.

Every other constructor simply passes its information to the master, which does the actual work.

ALL ROADS LEAD TO THE MASTER

PATH C: "Chenda"

→ Passes: "Chenda", 18 (default), "Undeclared" (default)

|

↓

PATH B: "Dara, Engineering"

→ Passes: "Dara", 18 (default), "Engineering"

|

|

↓

→ MASTER CONSTRUCTOR
Validates every value
Sets every attribute
Produces the valid object

Why the Master Constructor Matters

WITHOUT MASTER CONSTRUCTOR

Each path sets its own defaults

→ 3 places to update

Each path applies its own validation

→ 3 places to fix a bug

WITH MASTER CONSTRUCTOR

Only the master sets defaults

→ 1 place to update

Only the master validates

→ 1 place to fix a bug

updating every single path
→ easy to forget one

just update the master once
→ all paths benefit automatically

Chaining — How One Constructor Calls Another

*"In code, a convenience constructor can delegate to the master by passing along default values for whatever was not provided. This is called **constructor chaining** — one constructor hands off to another."*

CONCEPT:

Path C knows only the name: "Chenda"

It says: "I'll handle this – but I'll pass the rest to the
master using sensible defaults for what I don't know"
→ It calls: `master("Chenda", age=18, major="Undeclared")`

Path B knows name and major: "Dara", "Engineering"

→ It calls: `master("Dara", age=18, major="Engineering")`

The master does the actual validation and setup.
The other paths are just convenient shortcuts.

The Critical Rule for Chaining

"When one constructor hands off to another, that handoff must be the VERY FIRST thing it does — before anything else. The master must be set up completely before any other work begins."

RIGHT:

Path B calls master first
then any extra work
→ the master always runs first
→ always produces a valid object

WRONG:

Path B does other things first,
then calls master
→ master may run on partial info
→ unpredictable results

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading a Constructor as a Concept

```
// The MASTER CONSTRUCTOR - all setup logic lives here
public Student(String name, int age, String major) {
    setName(name);    // calls the setter from Week 4 - validation runs
    setAge(age);       // calls the setter - validation runs
    setMajor(major);   // calls the setter - validation runs
}

// A CONVENIENCE CONSTRUCTOR - only name and major provided
public Student(String name, String major) {
    this(name, 18, major); // hands off to master with a default age
}

// A CONVENIENCE CONSTRUCTOR - only name provided
public Student(String name) {
    this(name, 18, "Undeclared"); // hands off to master with two defaults
}

// A DEFAULT CONSTRUCTOR - nothing provided at all
public Student() {
    this("Unknown", 18, "Undeclared"); // hands off to master
}
```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
public Student(...)	"This is the setup action for a Student"
setName(name)	"Run the validation check, then save the name"
this(name, 18, major)	"Hand off to the master constructor with defaults"
new Student("Sokha", 20)	"Build a complete Student using the setup action"

The Before and After

```
// BEFORE constructors (awkward, incomplete, easy to forget a step):
Student sokha = new Student(); // empty, invalid
sokha.setName("Sokha");
sokha.setAge(20);
sokha.setMajor("CS");
// Three steps. What if one is forgotten? Object is broken.
```

```
// AFTER constructors (complete, valid, one step):  
Student sokha = new Student("Sokha", 20, "CS");  
// Done. Validated. Ready. Cannot be broken by a forgotten step.
```

Test Checkpoint

- ☐ Can explain the "incomplete object" problem using the hotel analogy (or a different real-world example)
- ☐ Can explain what a constructor is in one sentence without using Java-specific words
- ☐ Can describe the difference between "full details" and "name only" constructor paths using the hotel/defaults analogy
- ☐ Can explain why the master constructor matters (one place to update, one place to validate)
- ☐ Can explain why the handoff to the master must be the first action in any convenience constructor

Common Misconceptions

Misconception	Clarification
"A constructor is just a method like any other"	A constructor is a special one-time setup action — it runs automatically on creation, cannot be called again, and has no return type.
"I can set things up after creation and it's the same"	No — it leaves an incomplete object existing in the system. Like getting a hotel key before check-in is complete.
"Defaults are just 'zeroes' or 'nulls'"	Defaults must be meaningful valid values — "Undeclared" for major, 18 for age. A null name or 0 age is not a valid default.
"Every constructor needs to do all the validation itself"	No — convenience constructors hand off to the master, which does all the validation. Validation is defined once, in one place.
"The handoff to the master can happen anywhere in the constructor"	No — it must always be the very first action. The master must run first, before anything else.

Extension Challenges

● Basic — Spot the Problem

Read this setup sequence and identify what can go wrong:

A hospital creates a Patient object, then sets the name, then sets the blood type, then forgets to set the date of birth. The patient record now exists in the system with no date of birth.

Answer these questions in plain English:

1. What is the incomplete object problem here?
2. What real-world harm could result from this incomplete record?
3. How would a constructor prevent this specific problem?

● Intermediate — Design the Setup Paths

Design the setup options (in plain English — no code needed) for a `LibraryMember` class:

- Attributes: `name`, `memberId`, `membershipType` ("Standard" / "Premium"), `booksAllowed`
- A Premium member is allowed 10 books; a Standard member is allowed 5
- Design three setup paths:
- Full path: name + memberId + membershipType
- Standard path: name + memberId only (membershipType defaults to "Standard")
- Staff path: name only (memberId auto-generated, membershipType = "Staff Premium")
- Identify: which path is the master? What defaults does each convenience path fill in?

● Advanced — Constructor Design Decisions

Consider two different designs for creating a `BankAccount` object:

Design A: One constructor requiring: `owner`, `initialBalance`, `accountType`, `branchCode`, `currency`

Design B: One master constructor requiring: `owner`, `accountType`, then sensible defaults for the rest

Write a short analysis:

- What is the advantage of Design A? What problem does it create for callers?
 - What is the advantage of Design B? What risk does it introduce with defaults?
 - When might Design A be safer? When might Design B be more practical?
 - How does the master constructor principle help manage the complexity of both?
-

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. The **PROBLEM**: creating an object in multiple separate steps leaves incomplete, invalid objects in the system.
Like a hotel handing over a key before check-in is done.
 2. A **CONSTRUCTOR** is a one-time setup action that runs automatically when an object is created.
The object is not available until setup is complete.
 3. **DEFAULTS** are meaningful fallback values for information not provided. They must be valid – not just "nothing."
 4. **OVERLOADING** means offering multiple setup paths – full details, partial details, or just a name.
All paths produce a complete, valid object.
 5. The **MASTER CONSTRUCTOR** principle: all setup logic lives in one place. Convenience paths hand off to the master with defaults filled in. One place to update means one place to fix and maintain.
 6. The handoff to the master must always be **FIRST** – before any other work in a convenience constructor.
 7. Constructors call the same setters defined in Week 4, so validation still runs – even during initial setup.
-

Homework

Continue your Smartphone design from Weeks 2–4. Apply constructors:

1. Design **three setup paths** for a `Smartphone` (plain English — no code needed):
 - Full path: `brand`, `model`, `storageGB` — all provided
 - Partial path: `brand` and `model` only — `storageGB` defaults to 64
 - Minimal path: `brand` only — `model` defaults to "Standard", `storageGB` defaults to 32
2. Identify which path is the **master** — the one where all validation and attribute-setting actually happens

4. **Before/After comparison:** Show the old multi-step setup (3+ setter calls) versus the new single-step setup. Which is safer? Why?

Optional bonus: Try writing these three paths as actual Java constructors, following today's illustration as a guide.

Bring your design to Week 6 — we will explore Inheritance, where one class builds on top of another. We will need to understand how setup works across parent-child class relationships.

Week 5 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com